Task Management API - Stage 1: Root & Health Endpoints

Overview

This repository contains a lightweight RESTful Task Management API built using Node.js and Express as part of the FlyRank Internship Backend Engineering Track (Week 2).

Stage 1 expands the core HTTP daemon by implementing structured routing rules (GET / and GET /health), establishing clean repository tracking using .gitignore, and enforcing standard JSON MIME content types (application/json).

Technical Stack

- **Runtime Environment:** Node.js (v24.x)
- **Framework:** Express.js
- **Protocol & Data Format:** HTTP/1.1 with JSON payload serialization
- **Shell Environment:** VS Code Integrated Terminal (Windows PowerShell)
- **Version Control:** Git (Local repository-scoped identity)

Directory & Environment Setup

1. **Workspace Navigation (VS Code Terminal):**
   Navigate into the designated project workspace (using double quotes to handle space-delimited directory paths):
   cd "C:\Users\ernes\Documents\Ernest Career\Training\FlyRank Internship\Week2\BackEnd Engineering\Build your first CRUD API\NodeJS"

2. **Repository Hygiene (.gitignore):**
   To prevent tracking volatile third-party dependency folders (node_modules), a .gitignore file is placed in the project root:
   node_modules/

   If node_modules/ was previously staged in Git tracking, remove it from the index while preserving the physical folder on disk:
   git rm -r --cached node_modules

API Endpoints & Routing Architecture

Method	Endpoint	Description	Windows

			Sysadmin Analogy
GET	/	Returns API identity metadata and endpoint catalog	**IIS Server Identity Banner / SNMP Discovery Query**
GET	/health	Returns service status ({"status":"ok"})	**System Heartbeat Probe / Load Balancer Health Check**

Execution & Daemon Lifecycle

To start the HTTP server daemon in the foreground:

node server.js

Expected Console Output:

Server is running and listening on http://localhost:3000

> **Note for Sysadmins:** When modifying server.js, changes saved to disk require restarting the active Node process (Ctrl + C then node server.js) to load the updated instructions into volatile system RAM.

Verification & Protocol Inspection

To test routing rules and verify status codes, open a secondary VS Code terminal tab and execute raw curl requests with HTTP header inspection enabled (-i):

1. Root Endpoint Check (GET /):

curl -i http://localhost:3000/

Expected HTTP Response Output:

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 58
ETag: W/"3a-MI5kMm1z/AHkXM8xo8cNqUThTqA"

Date: Wed, 22 Jul 2026 22:48:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"name":"Task API","version":"1.0","endpoints":["/tasks"]}

2. System Health Check (GET /health):

curl -i http://localhost:3000/health

Expected HTTP Response Output:

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 15
ETag: W/"f-VaSQ4oDUizbLZNAEkkN+sX+q3Sg"
Date: Wed, 22 Jul 2026 22:49:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"ok"}

Version Control Commit History

Verify local Git snapshot records using the condensed audit log command:

git log --oneline

Audit Trail:

15fe97e (HEAD -> master) Stage 1: root and health endpoints
f4a5e22 Stage 0: hello server